

TomatoGuard AI

Database Implementation Specification

PostgreSQL (Neon) — corrected schema, v1 scope, migration script

How to Use This Document

This is the authoritative database reference for TomatoGuard AI v1. It is built on top of the schema your team already deployed to Neon — not a redesign. Every change here is either (a) a correctness fix (missing constraints that could silently corrupt data) or (b) a v1 scope decision your team already made. If you're implementing backend or frontend work, treat the table/column names here as final and exact.

What changed from the schema your team deployed, and why

Change	Reason
Added PRIMARY KEY to every table	bigserial alone does not create a primary key — as deployed, Postgres would accept duplicate IDs
Added FOREIGN KEY constraints on every *_id reference column	As deployed, e.g. disease_scans.user_id could reference a user that doesn't exist — no referential integrity was enforced
Added UNIQUE constraint on users.email	Prevents two accounts registering the same email — not enforced in the original DDL
Added UNIQUE constraint on farms.user_id	Enforces the v1 decision: exactly one farm per farmer. Remove this single line to unlock multi-farm in v2 — no other schema change needed
Added image_public_id columns to disease_scans and soil_reports	Cloudinary returns both a secure_url and a public_id; you need the public_id stored to ever delete/replace an image later
Added indexes on all foreign key columns	Every dashboard/history query filters by user_id or farm_id — without an index these become full table scans as data grows
admin table: activated for v1, as a separate table (not a role field)	Your team's planning doc suggested role-based (users.role); the deployed SQL has a separate admin table instead. Decision: keep the separate table — it's already deployed and working, and a separate table cleanly isolates admin auth from farmer auth. users.admin_id stays unused (it implied per-farmer assigned admins, which v1's admin panel doesn't do — admin oversight is global, not per-farmer)
Added users.is_active	New column — backs the admin 'deactivate account' capability. Not in the original deployed schema

V1 active tables vs. deferred tables

Status	Tables
Active in v1	users, admin, farms, diseases, disease_scans, soil_reports, soil_questionnaires, soil_analyses, weather_records
Deferred to v2 (table exists, no endpoints, no migration fixes applied)	daily_tasks, notifications, government_schemes, reports
Not created (decided against for v1)	user_settings — Settings page stays frontend-only (localStorage) for now
Not created (v1 resolves this without a table)	recommendations — the combined DSS output is computed on-the-fly from the latest disease_scans + soil_analyses + weather_records rows, not persisted. See the Backend document, Phase 6.

On the deferred tables: `daily_tasks`, `notifications`, `government_schemes`, and `reports` remain in your Neon database exactly as your teammate created them. Do not run migrations against them yet and do not build endpoints for them in v1 — but do not drop them either, since your team's v2 plan already accounts for their exact shape.

1. Entity Relationship Summary (v1 active tables)

Entity	Related entity	Relationship
admin	diseases	manages (no FK — admin actions are audited via updated_at only in v1)
admin	users	oversees (no FK — admin views/deactivates any farmer, not linked to a specific one)
users	farms	1 : 1 (v1 — enforced by UNIQUE on farms.user_id)
users / farms	disease_scans	1 : Many
diseases	disease_scans	1 : Many (lookup table)
users / farms	soil_reports	1 : Many
soil_reports	soil_analyses	1 : Many (nullable FK — analysis may instead come from a questionnaire)
users / farms	soil_questionnaires	1 : Many
soil_questionnaires	soil_analyses	1 : Many (nullable FK)
users / farms	weather_records	1 : Many

2. Corrected Schema — Full Table Reference

2.1 — users

Column	Type	Constraints / Notes
user_id	bigserial	PRIMARY KEY
name	varchar(100)	NOT NULL
email	varchar(150)	NOT NULL, UNIQUE
password_hash	text	NOT NULL — bcrypt, never plaintext
phone	varchar(20)	nullable
address	text	nullable
village	varchar(100)	nullable
district	varchar(100)	nullable
state	varchar(100)	nullable
admin_id	bigint	nullable — UNUSED (see design note above)
is_active	boolean	NEW — default true. Set false by admin to deactivate an account
created_at	timestamptz	default now()
updated_at	timestamptz	default now()

2.0 — admin

Column	Type	Constraints / Notes
admin_id	bigserial	PRIMARY KEY
name	varchar(100)	NOT NULL
email	varchar(150)	NOT NULL, UNIQUE
password_hash	text	NOT NULL — bcrypt, same as users
phone	varchar(20)	nullable
created_at	timestamptz	default now()

Auth isolation: admin and users are authenticated completely separately — a JWT issued for an admin_id must never be accepted by a farmer-only endpoint, and vice versa. See Backend document, Admin Module, for how the token type claim enforces this.

No admin self-signup: there is no POST /admin/signup endpoint. Admin accounts are created directly in the database (a one-time INSERT, or a simple seed script) — a public signup form for admin accounts would be a real security hole in a student project with no additional vetting layer.

2.2 — farms (v1: exactly one row per user)

Column	Type	Constraints / Notes
farm_id	bigserial	PRIMARY KEY
user_id	bigint	FOREIGN KEY → users.user_id, NOT NULL, UNIQUE (v1

		one-farm rule)
farm_name	varchar(150)	NOT NULL
location	text	nullable
area_acres	numeric(10,2)	nullable
soil_type	varchar(100)	nullable
latitude	numeric(9,6)	nullable — used by weather calls
longitude	numeric(9,6)	nullable
created_at	timestamptz	default now()

v1 auto-creation rule: The backend creates one farms row automatically at signup (see Backend doc, Phase 1) — the farmer never sees a 'create farm' step in v1. This keeps the frontend flow identical to the original single-session design while the schema stays multi-farm-ready underneath.

2.3 — diseases (lookup table, pre-seeded, not user-writable)

Column	Type	Constraints / Notes
disease_id	bigserial	PRIMARY KEY
disease_name	varchar(150)	NOT NULL, UNIQUE
description	text	nullable
symptoms	text	nullable
causes	text	nullable
prevention	text	nullable
treatment	text	nullable
created_at	timestamptz	default now()
updated_at	timestamptz	default now()

Seed this table with the 10 PlantVillage tomato classes (Bacterial_spot, Early_blight, Late_blight, Leaf_Mold, Septoria_leaf_spot, Spider_mites, Target_Spot, Yellow_Leaf_Curl_Virus, mosaic_virus, healthy) before Phase 3 backend work starts — confirm the exact label strings match your trained model's output classes exactly, since disease_scans.predicted_disease must match disease_name for the disease_id lookup join to work.

2.4 — disease_scans

Column	Type	Constraints / Notes
scan_id	bigserial	PRIMARY KEY
user_id	bigint	FOREIGN KEY → users.user_id, NOT NULL
farm_id	bigint	FOREIGN KEY → farms.farm_id, nullable
disease_id	bigint	FOREIGN KEY → diseases.disease_id, nullable — null if predicted label doesn't match a seeded row
image_path	text	NOT NULL — Cloudinary secure_url
image_public_id	text	NEW — Cloudinary public_id, needed to delete/replace
predicted_disease	varchar(150)	NOT NULL — raw model output label

confidence	numeric(5,2)	CHECK 0–100
severity	varchar(30)	nullable
model_name	varchar(100)	default 'EfficientNetB0'
model_version	varchar(50)	nullable
recommendation	text	nullable — short immediate action, distinct from diseases.treatment
scan_date	timestampz	default now()

2.5 — soil_reports (uploaded file + raw OCR)

Column	Type	Constraints / Notes
soil_report_id	bigserial	PRIMARY KEY
user_id	bigint	FOREIGN KEY → users.user_id, NOT NULL
farm_id	bigint	FOREIGN KEY → farms.farm_id, nullable
report_file_path	text	Cloudinary secure_url
report_file_public_id	text	NEW — Cloudinary public_id
report_date	date	nullable
uploaded_at	timestampz	default now()

2.6 — soil_questionnaires

Column	Type	Constraints / Notes
questionnaire_id	bigserial	PRIMARY KEY
user_id	bigint	FOREIGN KEY → users.user_id, NOT NULL
farm_id	bigint	FOREIGN KEY → farms.farm_id, nullable
crop_stage	varchar(100)	nullable
previous_crop	varchar(100)	nullable
irrigation_type	varchar(100)	nullable
fertilizer_used	text	nullable
soil_color	varchar(100)	nullable
drainage_condition	varchar(100)	nullable
additional_answers	jsonb	nullable — free-form extra Q&A
submitted_at	timestampz	default now()

2.7 — soil_analyses (the result, from either input path)

Column	Type	Constraints / Notes
analysis_id	bigserial	PRIMARY KEY
user_id	bigint	FOREIGN KEY → users.user_id, NOT NULL

farm_id	bigint	FOREIGN KEY → farms.farm_id, nullable
soil_report_id	bigint	FOREIGN KEY → soil_reports.soil_report_id, nullable
questionnaire_id	bigint	FOREIGN KEY → soil_questionnaires.questionnaire_id, nullable
ph, nitrogen, phosphorus, potassium, moisture, organic_matter	numeric	all nullable — not every source fills every field
predicted_soil_condition	varchar(150)	nullable — only populated IF a trained soil model exists (unconfirmed, see Backend doc Phase 4)
confidence	numeric(5,2)	nullable, CHECK 0–100 — null when rule-based, not model-based
model_name	varchar(100)	nullable
model_version	varchar(50)	nullable
fertilizer_recommendation	text	nullable
irrigation_recommendation	text	nullable
analysis_date	timestampz	default now()

Application-layer rule: exactly one of soil_report_id / questionnaire_id should be non-null per row in practice (a given analysis comes from one path or the other) — enforce this in the backend service, not as a DB-level CHECK, to keep the migration low-risk.

2.8 — weather_records

Column	Type	Constraints / Notes
weather_id	bigserial	PRIMARY KEY
farm_id	bigint	FOREIGN KEY → farms.farm_id, NOT NULL
temperature_c	numeric(6,2)	nullable
humidity_percent	numeric(5,2)	nullable
rainfall_mm	numeric(8,2)	nullable
wind_speed_kmh	numeric(8,2)	nullable
pressure_hpa	numeric(8,2)	nullable
weather_condition	varchar(100)	nullable
recorded_at	timestampz	default now()

3. Migration Script

Run this against your existing Neon database. It is additive/corrective only — it does not drop or rename any existing table or column, so your teammate's already-inserted data (if any) is preserved. Run in a transaction; if any step fails, the whole migration rolls back.

```
BEGIN;

-- Primary keys
ALTER TABLE admin ADD CONSTRAINT admin_pkey PRIMARY KEY (admin_id);
ALTER TABLE users ADD CONSTRAINT users_pkey PRIMARY KEY (user_id);
ALTER TABLE farms ADD CONSTRAINT farms_pkey PRIMARY KEY (farm_id);
ALTER TABLE diseases ADD CONSTRAINT diseases_pkey PRIMARY KEY (disease_id);
ALTER TABLE disease_scans ADD CONSTRAINT disease_scans_pkey PRIMARY KEY (scan_id);
ALTER TABLE soil_reports ADD CONSTRAINT soil_reports_pkey PRIMARY KEY
(soil_report_id);
ALTER TABLE soil_questionnaires ADD CONSTRAINT soil_questionnaires_pkey PRIMARY KEY
(questionnaire_id);
ALTER TABLE soil_analyses ADD CONSTRAINT soil_analyses_pkey PRIMARY KEY (analysis_id);
ALTER TABLE weather_records ADD CONSTRAINT weather_records_pkey PRIMARY KEY
(weather_id);

-- Uniqueness
ALTER TABLE admin ADD CONSTRAINT admin_email_unique UNIQUE (email);
ALTER TABLE users ADD CONSTRAINT users_email_unique UNIQUE (email);
ALTER TABLE diseases ADD CONSTRAINT diseases_name_unique UNIQUE (disease_name);
ALTER TABLE farms ADD CONSTRAINT farms_user_id_unique UNIQUE (user_id); -- v1 one-
farm rule

-- Foreign keys
ALTER TABLE farms ADD CONSTRAINT farms_user_fk FOREIGN KEY (user_id) REFERENCES
users(user_id);
ALTER TABLE disease_scans ADD CONSTRAINT scans_user_fk FOREIGN KEY (user_id)
REFERENCES users(user_id);
ALTER TABLE disease_scans ADD CONSTRAINT scans_farm_fk FOREIGN KEY (farm_id)
REFERENCES farms(farm_id);
ALTER TABLE disease_scans ADD CONSTRAINT scans_disease_fk FOREIGN KEY (disease_id)
REFERENCES diseases(disease_id);
ALTER TABLE soil_reports ADD CONSTRAINT reports_user_fk FOREIGN KEY (user_id)
REFERENCES users(user_id);
ALTER TABLE soil_reports ADD CONSTRAINT reports_farm_fk FOREIGN KEY (farm_id)
REFERENCES farms(farm_id);
ALTER TABLE soil_questionnaires ADD CONSTRAINT quest_user_fk FOREIGN KEY (user_id)
REFERENCES users(user_id);
ALTER TABLE soil_questionnaires ADD CONSTRAINT quest_farm_fk FOREIGN KEY (farm_id)
REFERENCES farms(farm_id);
ALTER TABLE soil_analyses ADD CONSTRAINT analyses_user_fk FOREIGN KEY (user_id)
REFERENCES users(user_id);
ALTER TABLE soil_analyses ADD CONSTRAINT analyses_farm_fk FOREIGN KEY (farm_id)
REFERENCES farms(farm_id);
ALTER TABLE soil_analyses ADD CONSTRAINT analyses_report_fk FOREIGN KEY
(soil_report_id) REFERENCES soil_reports(soil_report_id);
ALTER TABLE soil_analyses ADD CONSTRAINT analyses_quest_fk FOREIGN KEY
(questionnaire_id) REFERENCES soil_questionnaires(questionnaire_id);
ALTER TABLE weather_records ADD CONSTRAINT weather_farm_fk FOREIGN KEY (farm_id)
REFERENCES farms(farm_id);

-- Clouinary support columns
ALTER TABLE disease_scans ADD COLUMN image_public_id text;
ALTER TABLE soil_reports ADD COLUMN report_file_public_id text;
```

```
-- Admin panel support
ALTER TABLE users ADD COLUMN is_active boolean NOT NULL DEFAULT true;

-- Indexes for the FK columns every query will filter on
CREATE INDEX idx_disease_scans_user ON disease_scans(user_id);
CREATE INDEX idx_disease_scans_farm ON disease_scans(farm_id);
CREATE INDEX idx_soil_reports_user ON soil_reports(user_id);
CREATE INDEX idx_soil_questionnaires_user ON soil_questionnaires(user_id);
CREATE INDEX idx_soil_analyses_user ON soil_analyses(user_id);
CREATE INDEX idx_weather_records_farm ON weather_records(farm_id);

COMMIT;
```

4. Seed Data — diseases table

Run once, before Phase 3 backend work. Confirm exact class label strings against your team's trained model output before inserting — these must match `disease_scans.predicted_disease` exactly for the lookup join to resolve.

```
INSERT INTO diseases (disease_name, description, symptoms, causes, prevention,
treatment) VALUES
('Tomato_Bacterial_spot', 'Bacterial infection of leaves and fruit', '...', '...',
'...', '...'),
('Tomato_Early_blight', 'Fungal disease, Alternaria solani', '...', '...', '...',
'...'),
('Tomato_Late_blight', 'Fungal/oomycete, Phytophthora infestans', '...', '...', '...',
'...'),
('Tomato_Leaf_Mold', 'Fungal, Passalora fulva', '...', '...', '...', '...'),
('Tomato_Septoria_leaf_spot', 'Fungal, Septoria lycopersici', '...', '...', '...',
'...'),
('Tomato_Spider_mites', 'Pest, two-spotted spider mite', '...', '...', '...', '...'),
('Tomato_Target_Spot', 'Fungal, Corynespora cassiicola', '...', '...', '...', '...'),
('Tomato_Yellow_Leaf_Curl_Virus', 'Viral, whitefly-transmitted', '...', '...', '...',
'...'),
('Tomato_mosaic_virus', 'Viral', '...', '...', '...', '...'),
('Tomato_healthy', 'No disease detected', '...', '...', '...', '...');
```

Fill in the ... fields with real content from your model documentation or agronomic references before this goes live — placeholder text here would surface directly to farmers via the Disease Detection page.

4B. Seed Data — first admin account

Create the first admin account directly via SQL (there is no public admin signup form — see Backend document). Generate a real bcrypt hash for the password before inserting; never insert a plaintext password even temporarily.

```
-- Generate the hash separately, e.g.:
-- python3 -c "from passlib.context import CryptContext;
print(CryptContext(schemes=['bcrypt']).hash('your-chosen-password'))"

INSERT INTO admin (name, email, password_hash)
VALUES ('Admin Name', 'admin@tomatoguard.app', '<paste the generated bcrypt hash
here>');
```

5. Acceptance Criteria

- Migration runs against the real Neon database with zero errors, inside a transaction.
- Attempting to insert a `disease_scans` row with a non-existent `user_id` fails with a foreign key violation (proves constraints are live).
- Attempting to insert a second `farms` row for the same `user_id` fails with a uniqueness violation (proves the v1 one-farm rule is enforced).
- `diseases` table has exactly the confirmed class count, each row's `disease_name` matching the model's output labels exactly.
- `admin` table has exactly one seeded account with a real bcrypt hash, not plaintext.
- `users.is_active` defaults to true for new signups and can be independently set false without affecting any other table (no cascading deletes).

- Every FK column used in a dashboard or history query has a corresponding index (verify with EXPLAIN ANALYZE on the actual queries once Phase 2 backend work exists).